

CAPSTONE PROJECT

APEX BANK

ONLINE BANKING PORTAL

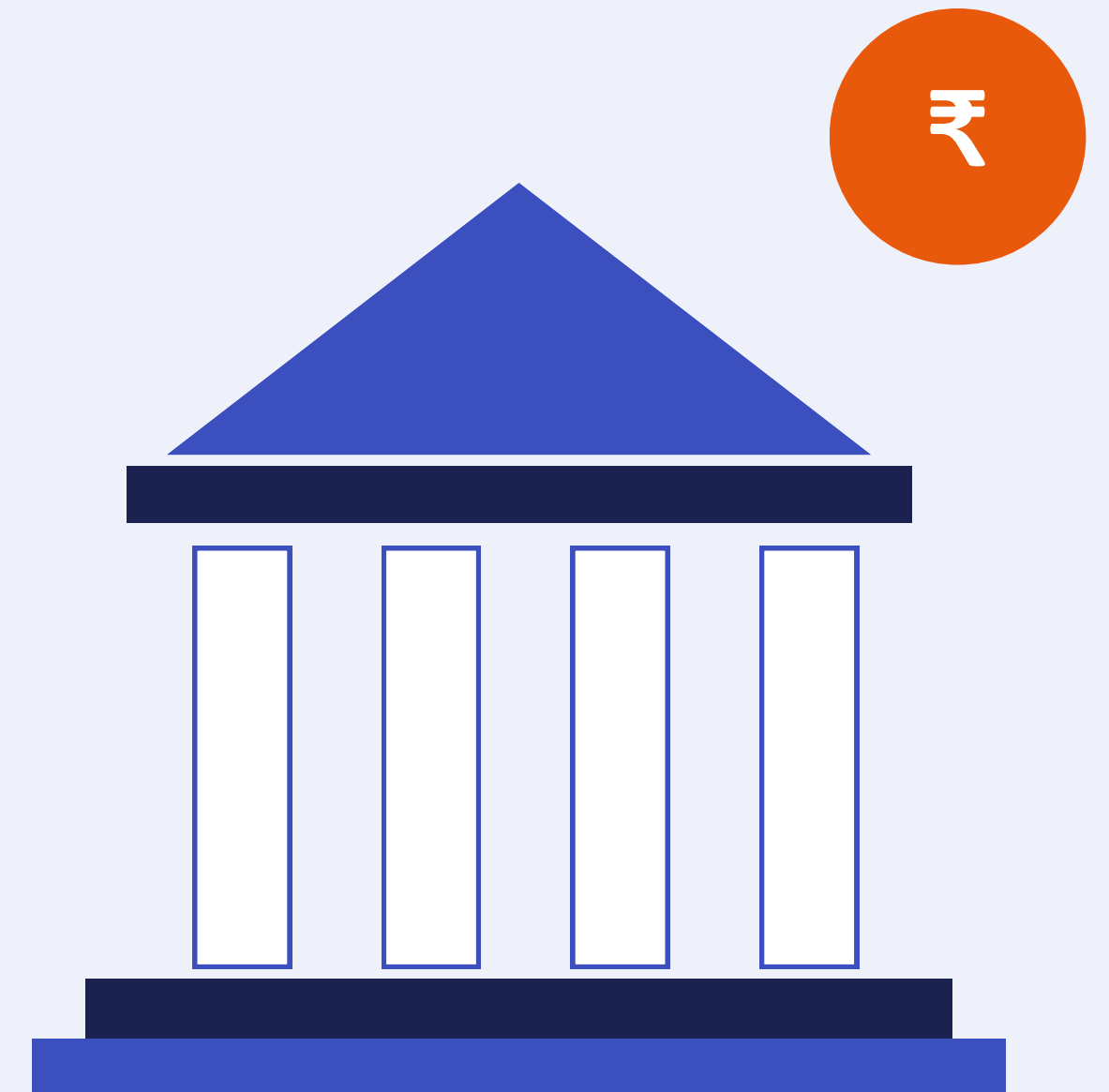
A full-stack online banking application built with a Spring Boot microservices backend and an Angular 17 frontend.

DEVELOPED BY

Ade Dikshith - 85010861

Nagam Srivatsav Reddy - 85010961

Indroji Sai Sushank - 85010926



Spring Boot · Angular 17 · MySQL 8

Outline

1 Introduction The ApexBank platform	2 Aim & Objectives What we set out to build	3 Functionalities Customer and admin features	4 System Requirements Hardware and software
5 Technologies Used Backend, frontend, tools	6 Project Modules Services and users	7 Architecture Design, security, APIs	8 Code Implementation Key code, explained
9 Testing & CI/CD Unit tests, Jenkins	10 Application Screenshots Customer and admin walkthrough	11 Future Scope Planned enhancements	12 Q&A Questions and discussion

Introduction

What is ApexBank?

- A modern, full-stack online banking application
- Simulates real-world digital banking operations
- Spring Boot microservices backend with an Angular 17 frontend

What it enables

- Customers open accounts and manage their profiles
- Transfer funds securely from anywhere
- Replaces branch-dependent banking with a fast, self-service platform

Industry-standard design

- Microservices with service discovery and an API Gateway
- JWT-based secure authentication
- Modern, responsive user interface

Real banking workflows

- Registration, OTP verification and internet-banking activation
- Admin-approved account opening
- NEFT and UPI transfers with transaction history

Aim of the Project

1

Secure, Scalable Banking

Design and develop a secure, scalable online banking system that mirrors real-world banking operations.

2

Seamless Customer Experience

Give customers a seamless digital experience for account management and fund transfers.

3

Centralised Admin Control

Give administrators a centralised dashboard to manage users, approvals and transactions.

4

Microservices in Practice

Demonstrate a practical implementation of microservices architecture in a real-world use case.

Objectives

01

Secure Authentication

Implement registration, login and authentication using JWT.

02

Admin-Approved Accounts

New customers apply for an account, subject to Admin approval.

03

Personalised Dashboard

Account balance, UPI ID and recent activity at a glance.

04

NEFT & UPI Transfers

Fund transfers with real-time balance updates.

05

Filterable Statements

A detailed, filterable statement of all transactions.

06

Dedicated Admin Portal

Account approvals and user management in one place.

07

Microservices Principles

Scalability, fault isolation and independent deployment.

5 backend services · 3 databases · 1 Angular app

Eureka, API Gateway, Auth, Account, Transaction

Functionalities of the Project

Authentication & Access

- Registration and login with account lockout
- OTP verification; forgot user ID / password
- Change password and secure logout
- JWT-based stateless sessions

Accounts & Dashboard

- Account opening requests, approved by Admin
- Dashboard: balance, UPI ID, recent transactions
- Account summary and date-range statement
- Profile management (view and update)

Payments

- Fund transfer via NEFT (account number)
- UPI transfer (UPI ID or account number)
- Payee management: save and delete beneficiaries
- Transaction password confirms each transfer

Admin Portal

- Approve or reject account requests
- User management: view all users
- Credit money to customer accounts
- Monitor all transactions system-wide

System Requirements

Hardware

- Processor: Intel i3 or above
- RAM: 8 GB minimum
- Storage: 10 GB free disk space

Software

- OS: Windows 10/11, macOS or Linux
- Java Development Kit (JDK) 17
- Node.js 18+ and Angular CLI 17
- MySQL Server 8.0
- IDE: Eclipse (backend), VS Code (frontend)
- Browser: Chrome, Edge, etc.

Run Prerequisites

- Java 17+
- Node.js 18+ and npm
- Docker and Docker Compose

Technologies Used

SERVER SIDE

Backend

- Spring Boot 3.3.2, Spring Cloud 2023.0.3
- Java 17, Maven (multi-module)
- Spring Data JPA (Hibernate)
- Spring Security + BCrypt
- JWT — jjwt 0.12.5
- MySQL 8 (database per service)
- Netflix Eureka — service discovery
- Spring Cloud Gateway — API Gateway
- OpenFeign — inter-service calls
- Lombok, Springdoc OpenAPI (Swagger UI)

CLIENT SIDE

Frontend

- Angular 17 (standalone components)
- TypeScript, RxJS, Angular signals
- Lazy-loaded routes
- Reactive Forms & validators
- HTTP interceptors
- Route guards
- Plain CSS — custom UI styling

TOOLS

Build & Run

- Maven, Git, Postman
- Docker & Docker Compose

TESTING

Quality & CI/CD

- JUnit 5
- Jenkins pipeline

Project Modules

AUTH SERVICE · 8081

Authentication

Registration, login, OTP, JWT issuing, password management.

ACCOUNT SERVICE · 8082

Account Management

Account opening requests, admin approval, profile, dashboard.

TRANSACTION SERVICE · 8084

Transactions

NEFT / UPI fund transfers, payees, transaction statements.

ADMIN PORTAL

Admin Module

Account approvals, user management, fund credit, system-wide monitoring.

GATEWAY · 8080

API Gateway

Single entry point, request routing, JWT validation.

EUREKA · 8761

Service Registry

Dynamic service discovery across all microservices.

ANGULAR 17 · 4200

Frontend (Angular)

Unified user interface for customers and admins.

OPENFEIGN

Inter-Service Calls

Services call one another with OpenFeign, resolved through Eureka.

User Characteristics

ROLE: CUSTOMER

Customers

- General public users with basic computer / smartphone literacy
- Need a simple, intuitive interface for banking tasks

ROLE: ADMIN

Administrators

- Bank staff who verify account requests, manage users and monitor transactions
- Need access to a centralised control panel

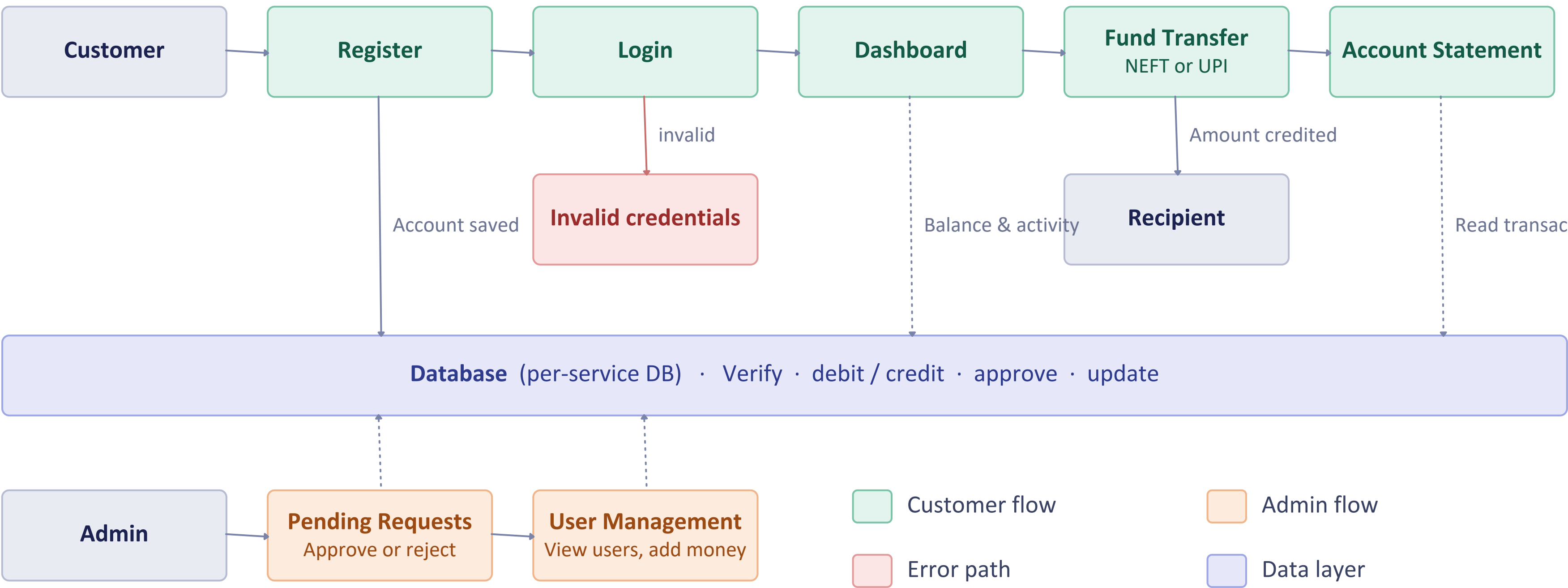
System expects users to have

- Basic internet familiarity
- A valid mobile number / email for OTP verification
- Access to a modern web browser

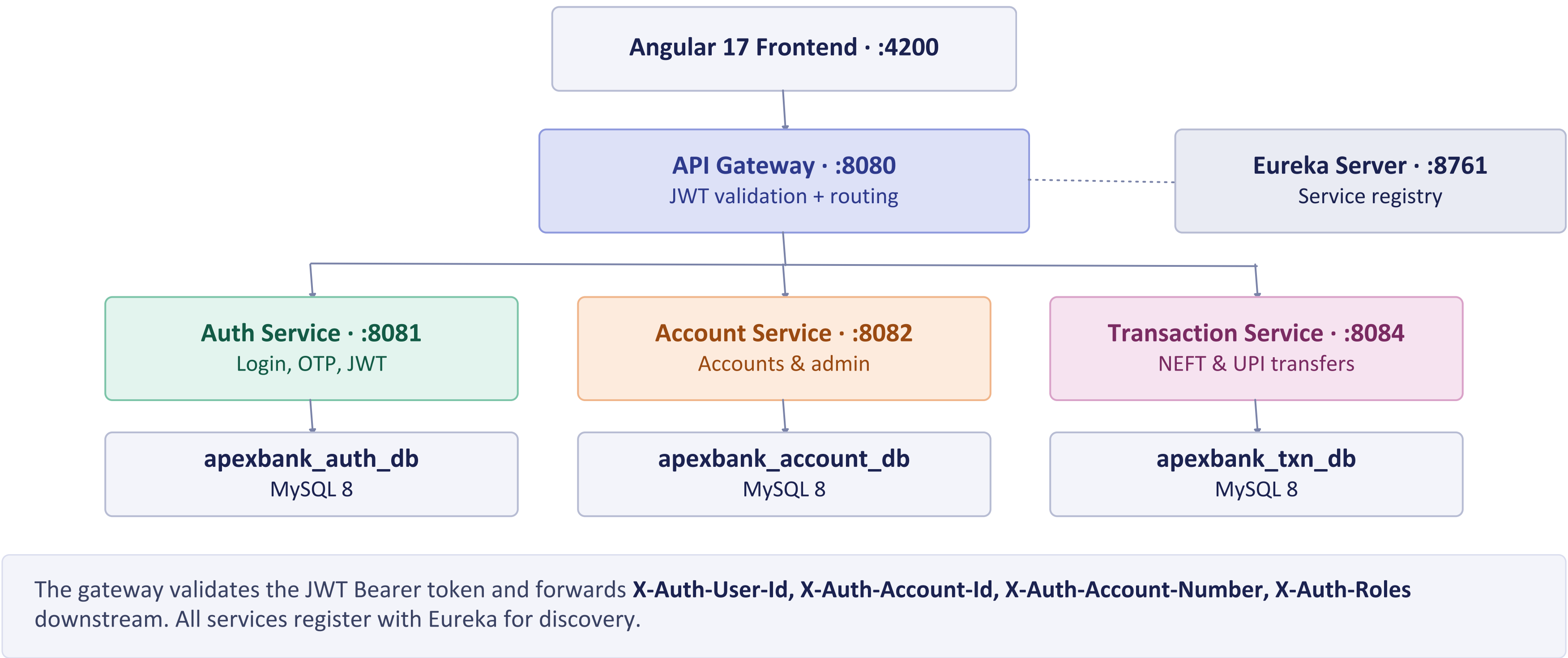
No technical expertise required

- Customers need no technical background
- Self-explanatory, beginner-friendly design

Sequence Diagram



Project Architecture

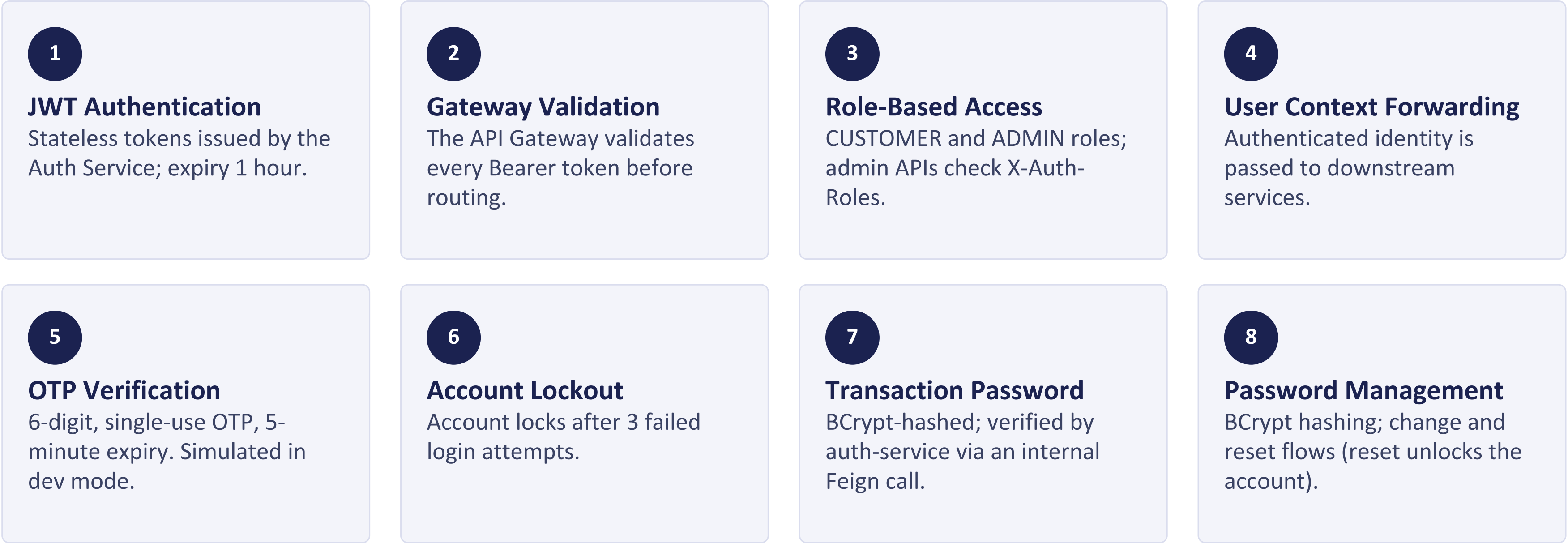


Microservices & Databases

Service	Port	Responsibilities	Database
Eureka Server	8761	Service registry and service discovery	—
API Gateway	8080	Single entry point, request routing, JWT validation, forwards authenticated user context	—
Auth Service	8081	Registration, login, OTP, password management, JWT issuing, account logout	apexbank_auth_db
Account Service	8082	Dashboard, account summary, profile, account opening, admin approvals, credit operations, system statistics	apexbank_account_db
Transaction Service	8084	NEFT, UPI, fund transfers, payees / beneficiaries, transaction history, statements	apexbank_txn_db

Database per service: each microservice owns its own MySQL schema; cross-service calls go through OpenFeign.

Security Architecture



Forwarded headers: **X-Auth-User-Id** · **X-Auth-Account-Id** · **X-Auth-Account-Number** · **X-Auth-Roles**

API Endpoints — Auth & Account

Auth Service

Port 8081 · reached through the API Gateway (8080)

- POST** `/api/auth/login`
Authenticate and issue JWT
- POST** `/api/auth/register`
Create internet-banking credentials
- POST** `/api/auth/send-otp`
Send a 6-digit OTP
- POST** `/api/auth/forgot-user-id`
Recover user ID
- POST** `/api/auth/forgot-password/validate-otp`
Validate the reset OTP
- POST** `/api/auth/set-new-password`
Set a new password
- POST** `/api/auth/change-password`
Change login / transaction password

Account Service — Dashboard & Admin

Port 8082 · reached through the API Gateway (8080)

- GET** `/api/dashboard`
Customer dashboard overview
- GET** `/api/dashboard/account-summary`
Account summary
- POST** `/api/accounts/open`
Open-account request (public)
- GET** `/api/admin/account-requests`
List requests (Admin)
- POST** `/api/admin/account-requests/{id}/approve`
Approve a request
- POST** `/api/admin/account-requests/{id}/reject`
Reject a request
- POST** `/api/admin/accounts/credit`
Credit an account (Admin)

API Endpoints — Transactions & Routes

Transaction Service

Port 8084 · through the API Gateway

- POST** `/api/fund-transfer/neft`
NEFT transfer by account number
- POST** `/api/fund-transfer/upi`
UPI by UPI ID or account number
- GET** `/api/fund-transfer/recent`
Recent transactions
- POST** `/api/fund-transfer/statement`
Transaction statement
- GET** `/api/payees`
List saved payees
- POST** `/api/payees`
Add a payee
- DELETE** `/api/payees/{id}`
Remove a payee

Customer

Angular frontend routes

- `/login`
- `/register`
- `/forgot-password`
- `/dashboard`
- `/dashboard/account-summary`
- `/dashboard/account-statement`
- `/dashboard/profile`
- `/dashboard/change-password`
- `/fund-transfer`
- `/payees`

Admin

Angular frontend routes

- `/admin/login`
- `/admin/dashboard`
- `/admin/requests`
- `/admin/users`
- `/admin/transactions`
- `/admin/credit`

Project Structure & Running the Project

PROJECT STRUCTURE

```
apexbank/  
├── Jenkinsfile  
├── backend/  
│   ├── pom.xml  
│   ├── docker-compose.yml  
│   ├── eureka-server/  
│   ├── api-gateway/  
│   ├── auth-service/  
│   ├── account-service/  
│   └── transaction-service/  
└── frontend/  
    └── src/  
        └── app/  
            ├── auth/  
            ├── dashboard/  
            └── admin/
```

PREREQUISITES

Java 17+ · Maven · Node 18+ & npm · Docker Compose

RUN THE APPLICATION

BACKEND

```
cd backend  
mvn clean package -DskipTests  
docker-compose up --build
```

FRONTEND

```
cd frontend  
npm install  
ng serve
```

LOCAL URLS

Frontend	http://localhost:4200
API Gateway	http://localhost:8080
Eureka	http://localhost:8761

Code: API Gateway — JWT Validation

api-gateway · JwtAuthGatewayFilter.java (excerpt)

@Component

```
public class JwtAuthGatewayFilter implements GlobalFilter, Ordered {
    public Mono<Void> filter(ServerWebExchange exchange,
                           GatewayFilterChain chain) {
        String path = exchange.getRequest().getURI().getPath();
        boolean isPublic = PUBLIC_PATHS.stream().anyMatch(path::startsWith);
        if (isPublic) return chain.filter(exchange); // login, register, OTP
        String authHeader =
            exchange.getRequest().getHeaders().getFirst("Authorization");
        if (authHeader == null || !authHeader.startsWith("Bearer "))
            return unauthorized(exchange); // 401
        try {
            Claims claims = Jwts.parser().verifyWith(key).build()
                .parseSignedClaims(authHeader.substring(7)).getPayload();
            ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
                .header("X-Auth-User-Id", claims.getSubject())
                .header("X-Auth-Roles", roles.toString())
                .build(); // + X-Auth-Account-Id, X-Auth-Account-Number
            return chain.filter(exchange.mutate().request(mutatedRequest).build());
        } catch (Exception ex) { return unauthorized(exchange); }
    }
}
```

WHAT IT DOES

What it does

- Global filter that runs on every request (order -1)
- Public paths skip the check
- Verifies the JWT signature ; bad or missing token returns 401
- Forwards identity as X-Auth-* headers

Code: Login, Account Lockout & JWT

auth-service · AuthService.java + JwtTokenProvider.java (excerpt)

```
@Transactional
public LoginResponse login(LoginRequest request) {
    User user = userRepository.findByUserId(request.getUserId())
        .orElseThrow(() -> new InvalidCredentialsException(
            "Invalid User ID or Password"));
    if (Boolean.TRUE.equals(user.getAccountLocked()))
        throw new AccountLockedException("Account locked ...");
    if (!passwordEncoder.matches(request.getPassword(),
                                user.getLoginPasswordHash())) {
        handleFailedLogin(user); // attempts + 1, lock at max (3)
        throw new InvalidCredentialsException("Invalid User ID or Password");
    }
    user.setFailedLoginAttempts(0); // reset on success
    String token = jwtTokenProvider.generateToken(authentication);
}

// JwtTokenProvider.generateToken(...)
return Jwts.builder()
    .subject(principal.getUsername())
    .claim("accountId", principal.getAccountId())
    .claim("accountNumber", principal.getAccountNumber())
    .claim("roles", roles)
    .issuedAt(now).expiration(expiry) // 3600000 ms = 1 hour
    .signWith(key).compact();
```

WHAT IT DOES

What it does

- BCrypt password check
- Wrong password adds 1 to the failed count; the 3rd failure locks the account
- Success resets the count and issues a 1-hour JWT carrying accountId and roles
- Reset-password flow unlocks the account

Code: Fund Transfer — Locking & Compensation

transaction-service · FundTransferService.java (excerpt)

```
@Transactional
public TransferSuccessResponse transferNeft(Long fromAccountId,
    String userId, NeftTransferRequest request) {
    verifyTransactionPassword(userId, request.getTransactionPassword());
    AccountInternalDto fromAccount = getAccountById(fromAccountId);
    AccountInternalDto toAccount =
        getAccountByNumber(request.getToAccountNumber());
    // rejects transfer to your own account
    debitThenCredit(fromAccount.id(), toAccount.id(), request.getAmount());
    // ... save Transaction (mode "NEFT", status "SUCCESS")
}

private void debitThenCredit(Long fromAccountId, Long toAccountId,
    BigDecimal amount) {
    accountClient.debit(fromAccountId, new DebitCreditRequest(amount));
    try {
        accountClient.credit(toAccountId, new DebitCreditRequest(amount));
    } catch (Exception ex) {
        // Compensating transaction: refund the source account
        accountClient.credit(fromAccountId, new DebitCreditRequest(amount));
        throw new IllegalStateException("Transfer failed while crediting "
            + "the beneficiary. Your amount has been refunded.", ex);
    }
}
```

account-service · row lock on debit (excerpt)

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findByIdForUpdate(Long id);

// InternalAccountService.debit(...)
Account account = accountRepository
    .findByIdForUpdate(accountId)
    .orElseThrow(...);
if (account.getBalance().compareTo(amount) < 0)
    throw new InsufficientBalanceException(...);
```

Testing & CI/CD Pipeline

SERVER SIDE

Backend Unit Tests

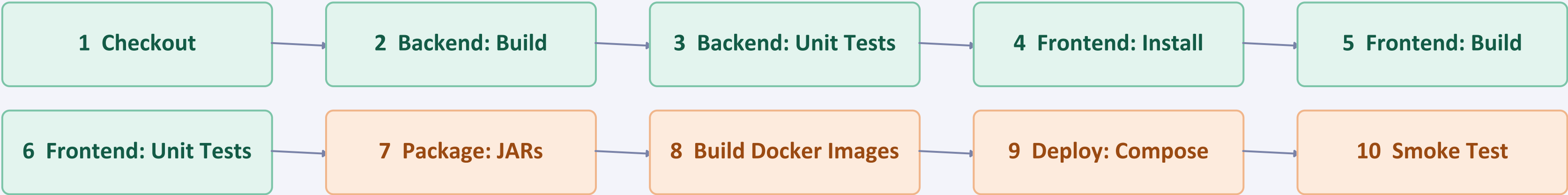
- 15 tests · JUnit 5 + Mockito · 3 services
- Login: 3 failed attempts lock the account
- Transfer: wrong txn password, self-transfer rejected
- Saga: credit failure refunds the source account
- Approval: request becomes an ACTIVE account

CLIENT SIDE

Frontend Unit Tests

- 26 specs · Jasmine + Karma · 4 spec files
- Admin login component and admin auth service
- Guards: guestGuard and adminAuthGuard
- CI runs ng test headless (ChromeHeadless)

JENKINSFILE — DECLARATIVE PIPELINE



Green = continuous integration · Orange = delivery (JARs, Docker images, Compose deploy)

Project Screenshots

A walkthrough of the 20 application screens — public site, customer portal and admin portal.

SLIDES 23–28

Public & Onboarding

- Home Page
- Dark Theme
- Open a New Savings Account
- Creating Internet Banking
- Sign In / Login Page

SLIDES 29–36

Customer Portal

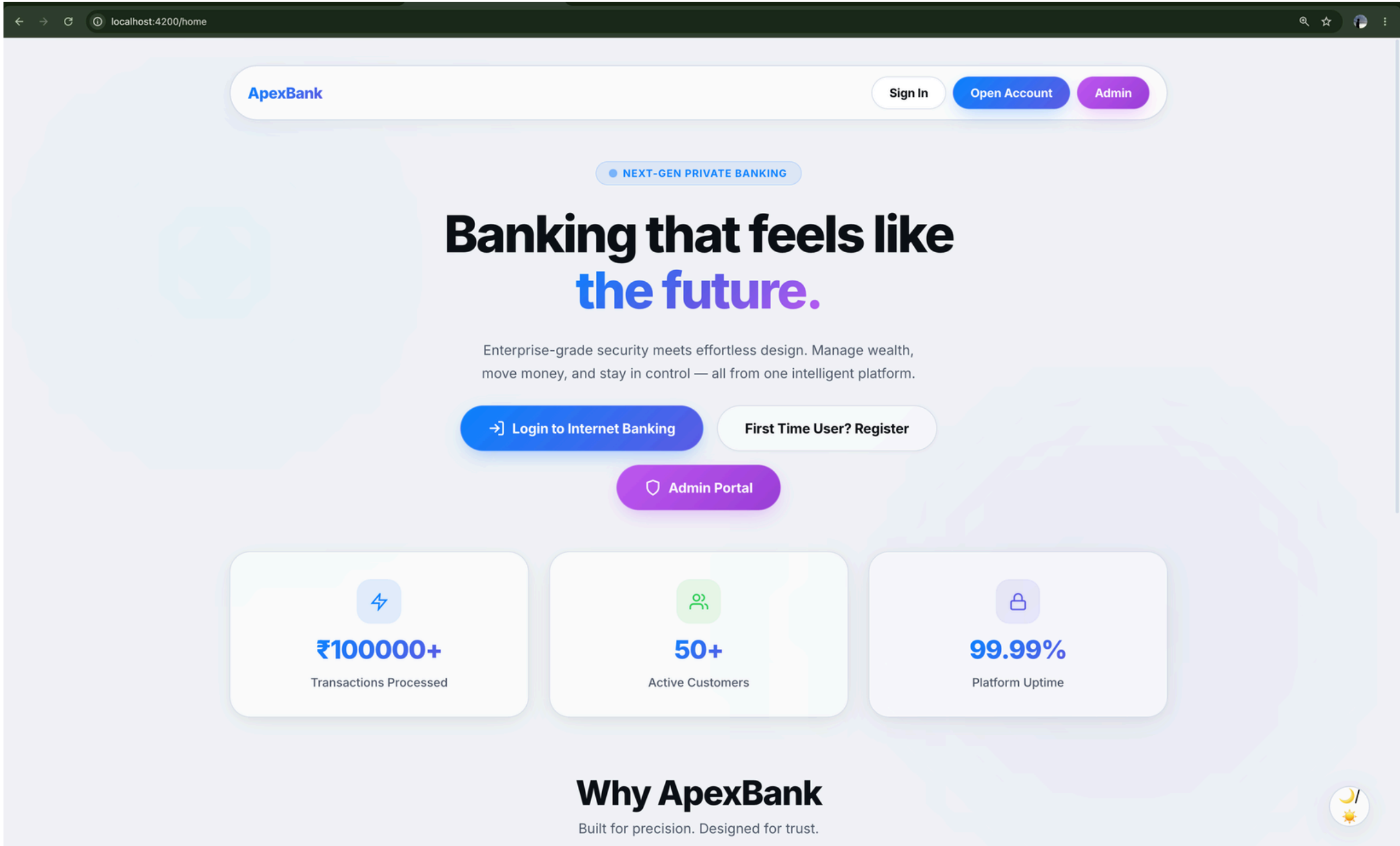
- User Dashboard
- Accounts Page
- Accounts Statement Page
- Fund Transfer Page
- Transaction Success Page
- Add Beneficiary Page
- User Profile Page
- Change Password Page

SLIDES 39–43

Admin Portal

- Admin Login Page
- Admin Dashboard Page
- Account Requests Page
- User Details Page
- New Users Amount Credit
- Transactions Tracking Page
- Logout Page

Home Page



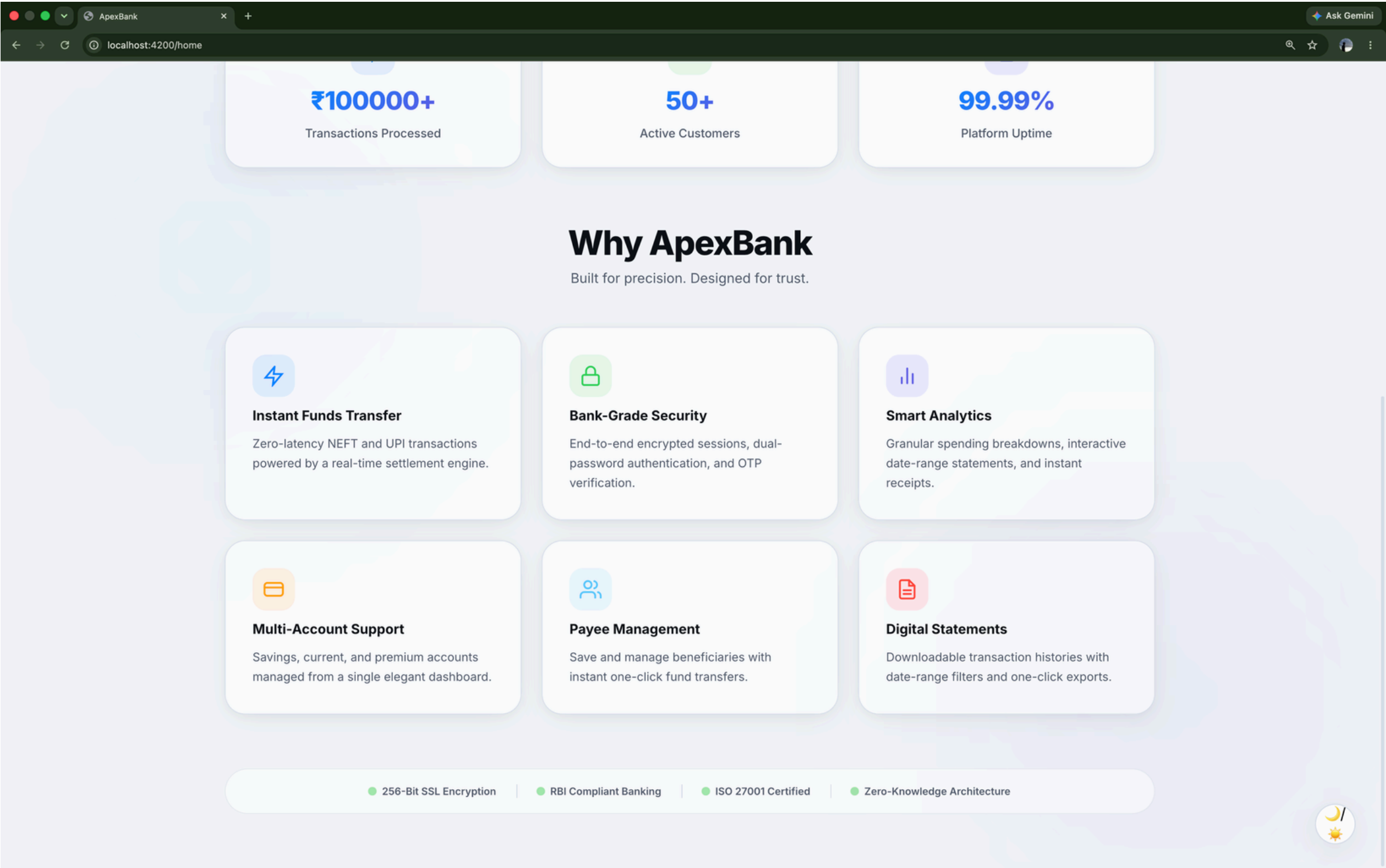
Screen Highlights

- Landing page with Login, Register and Admin Portal entry points
- Headline statistics and a “Why ApexBank” feature section
- Light theme shown; dark theme on the next slide

FRONTEND ROUTE

/

Home Page



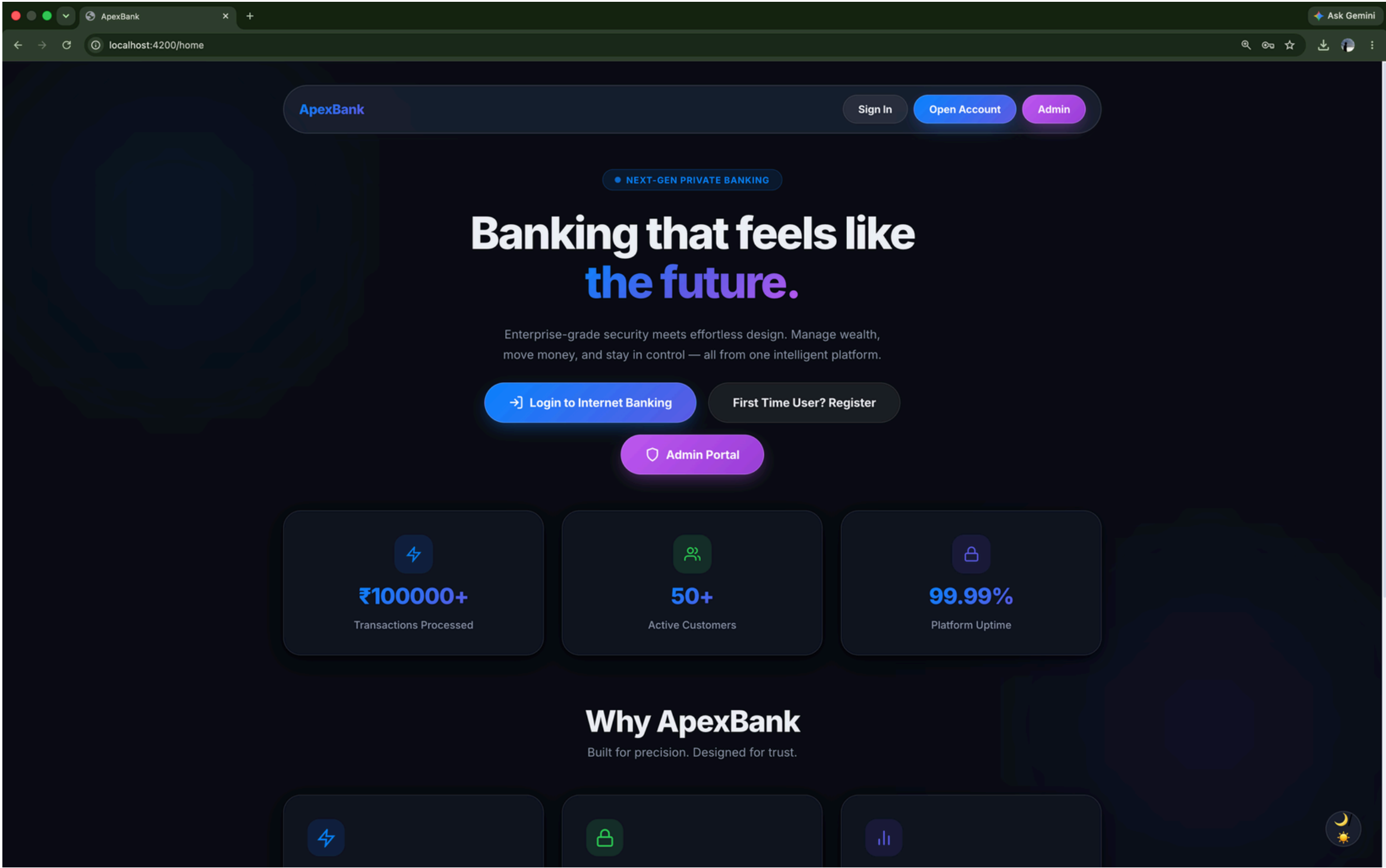
Screen Highlights

- Landing page with Login, Register and Admin Portal entry points
- Headline statistics and a “Why ApexBank” feature section
- Light theme shown; dark theme on the next slide

FRONTEND ROUTE

/

Dark Theme



Screen Highlights

- The same home page rendered in the dark theme
- Theme toggle sits at the bottom-right corner
- Readable contrast on both themes

Open a New Savings Account

ApexBank

Open a Savings Account

Complete the form below to apply for a new ApexBank account

Personal Details

TITLE

Mr

FIRST NAME

First name

MIDDLE NAME

Middle name (optional)

LAST NAME

Last name

FATHER'S NAME

Father's full name

MOBILE NUMBER

10-digit mobile number

EMAIL ID

email@example.com

AADHAR CARD NUMBER

12-digit Aadhar number

DATE OF BIRTH

dd/mm/yyyy

Residential Address

ADDRESS LINE 1

House/Flat No., Building Name

ADDRESS LINE 2

Street, Area, Locality

ADDRESS LINE 2

Street, Area, Locality

LANDMARK

Nearby landmark

STATE

State

CITY

City

PINCODE

6-digit pincode

Occupation Details

OCCUPATION TYPE

Select occupation

SOURCE OF INCOME

e.g. Salary, Business

GROSS ANNUAL INCOME

Select income range

Debit Card & Net Banking

☒ I want a Debit/ATM Card

☒ Opt for Net Banking

☐ I agree to the Terms & Conditions

Submit Application

[← Back to Home](#)

- Screen Highlights
- Personal details, residential address and occupation sections
 - Account preference and consent options
 - Submit Application sends the request for Admin approval

Creating Internet Banking

The screenshot shows a web browser window with the URL `localhost:4200/register`. The page displays the ApexBank logo and the title "Create Internet Banking Access". Below the title is a subtitle "Set up your online banking credentials". The form consists of several input fields and buttons:

- ACCOUNT NUMBER:** A single input field with the placeholder text "Enter your account number".
- LOGIN PASSWORD:** An input field with the placeholder text "Minimum 8 charact".
- CONFIRM LOGIN PASSWORD:** An input field with the placeholder text "Re-enter login pass".
- TRANSACTION PASSWORD:** An input field with the placeholder text "Minimum 6 charact".
- CONFIRM TRANSACTION PASSWORD:** An input field with the placeholder text "Re-enter transactio".
- OTP VERIFICATION:** An input field with the placeholder text "6-digit OTP" and a "Send OTP" button.
- Activate Internet Banking:** A large blue button.
- Already registered? Login:** A link at the bottom left.

Screen Highlights

- Account number with login and transaction passwords
- OTP verification using the Send OTP button
- Activate Internet Banking completes the setup

FRONTEND ROUTE

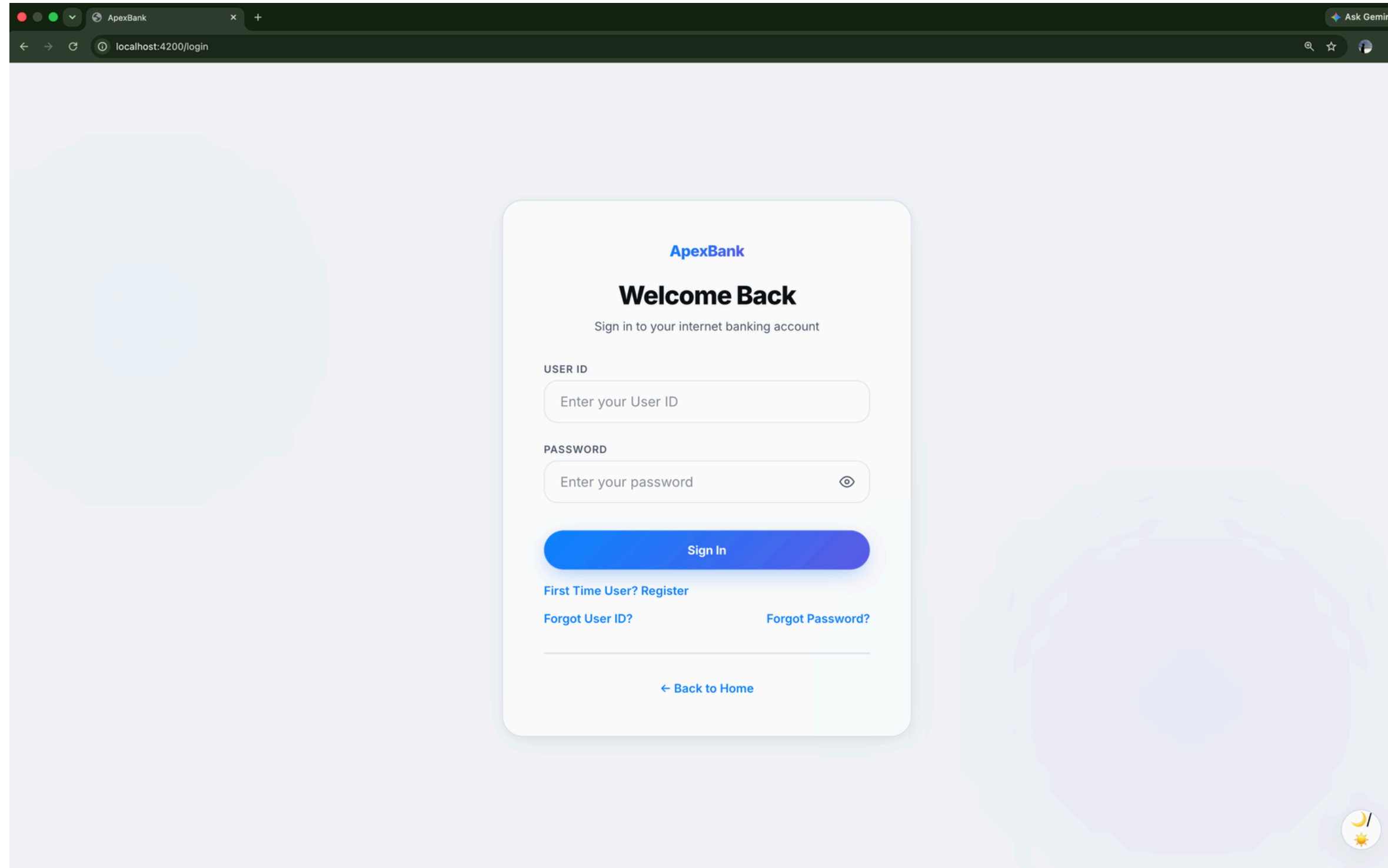
`/auth/register`

API ENDPOINTS

`POST /api/auth/send-otp`

`POST /api/auth/register`

Sign In / Login Page



Screen Highlights

- User ID and password with a show / hide toggle
- Links for registration, forgot user ID and forgot password
- Account locks after 3 failed attempts

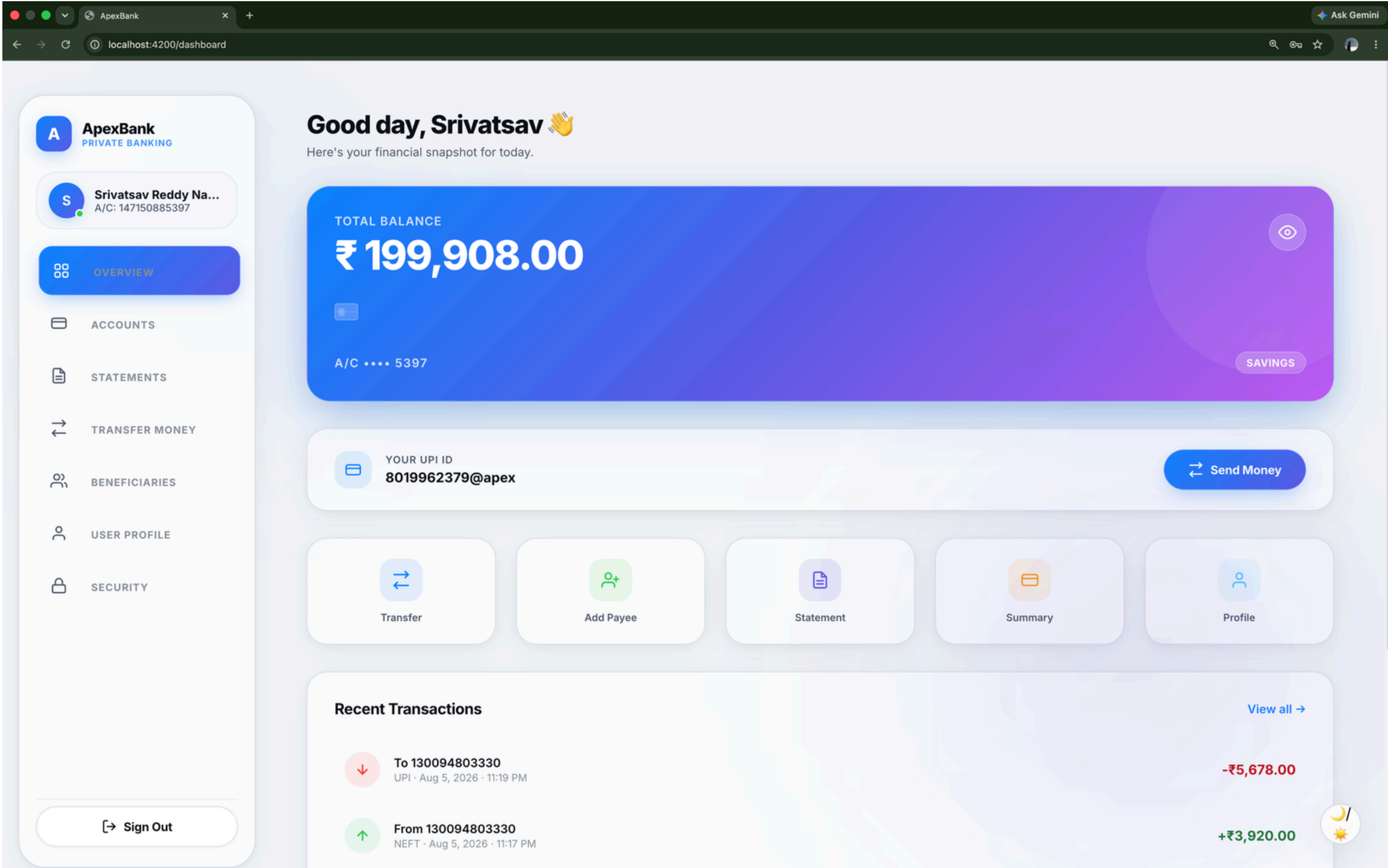
FRONTEND ROUTE

`/auth/login`

API ENDPOINTS

`POST /api/auth/login`

User Dashboard

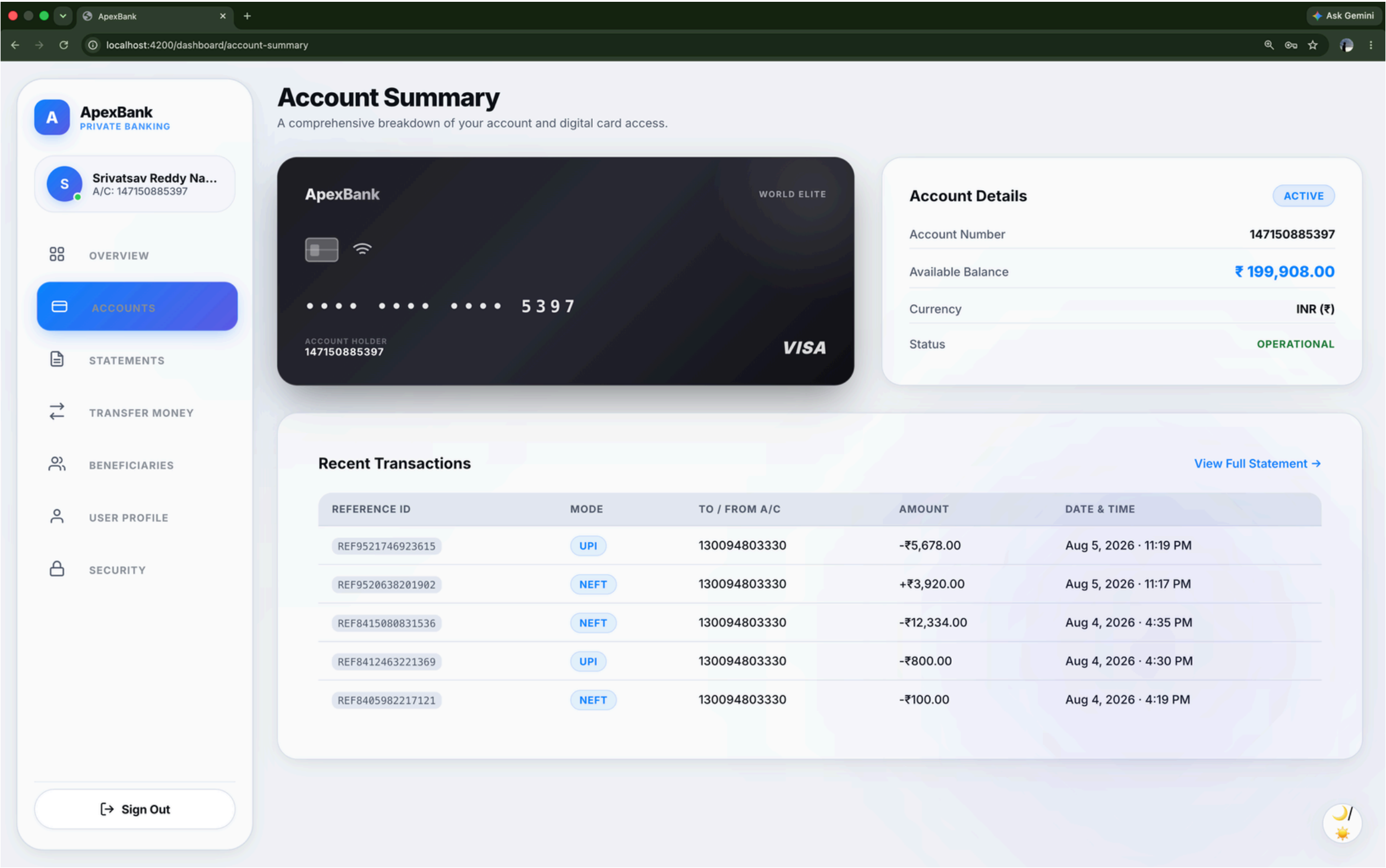


Screen Highlights

- Total balance card with masked account number
- UPI ID with a Send Money shortcut
- Quick actions and recent transactions

FRONTEND ROUTE
/dashboard
API ENDPOINTS
GET /api/dashboard

Accounts Page

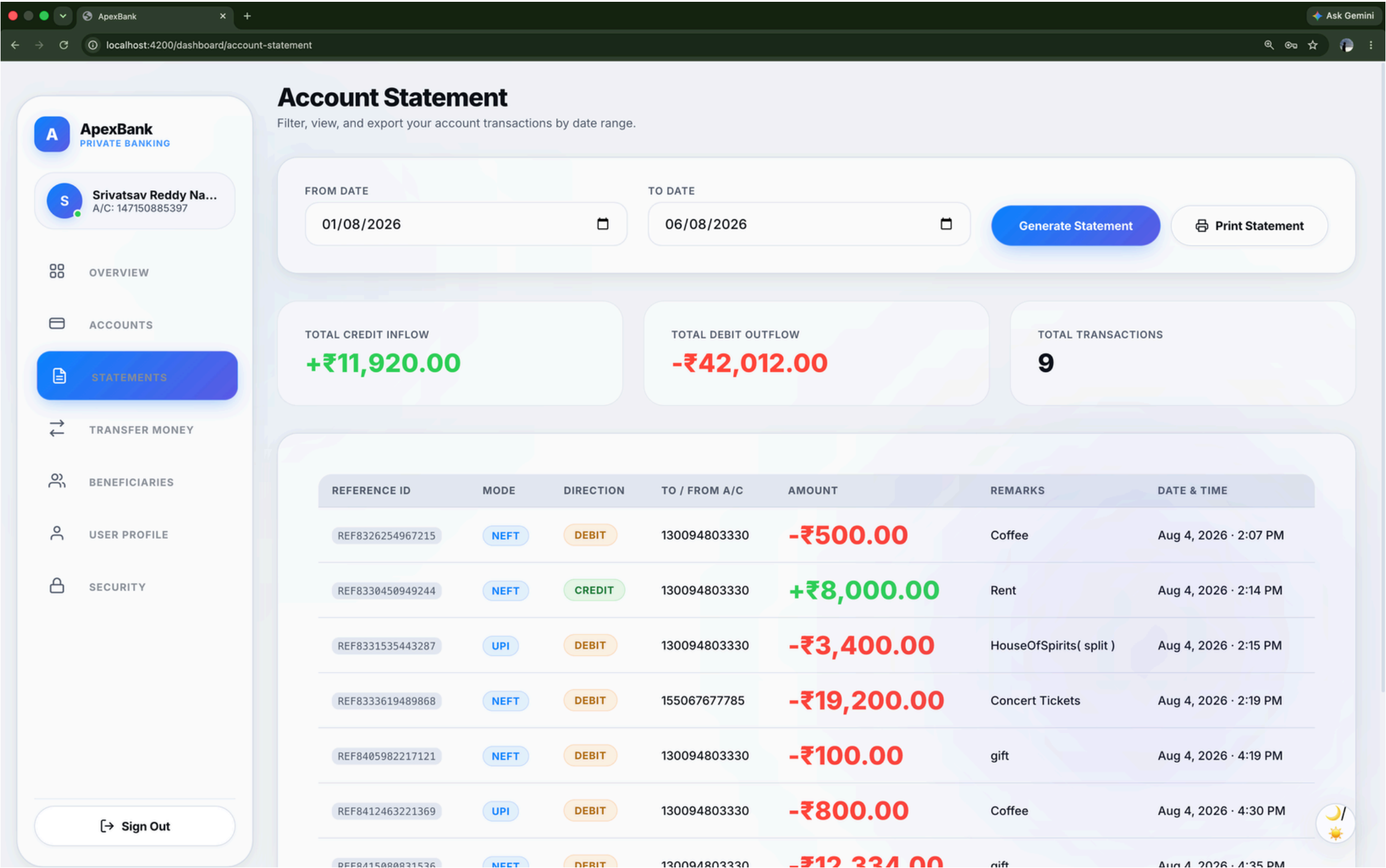


Screen Highlights

- Account details: number, balance, currency, status
- Card-style view of the account
- Recent transactions with reference ID and mode

FRONTEND ROUTE
`/dashboard`
API ENDPOINTS
`GET /api/accounts/summary`

Accounts Statement Page



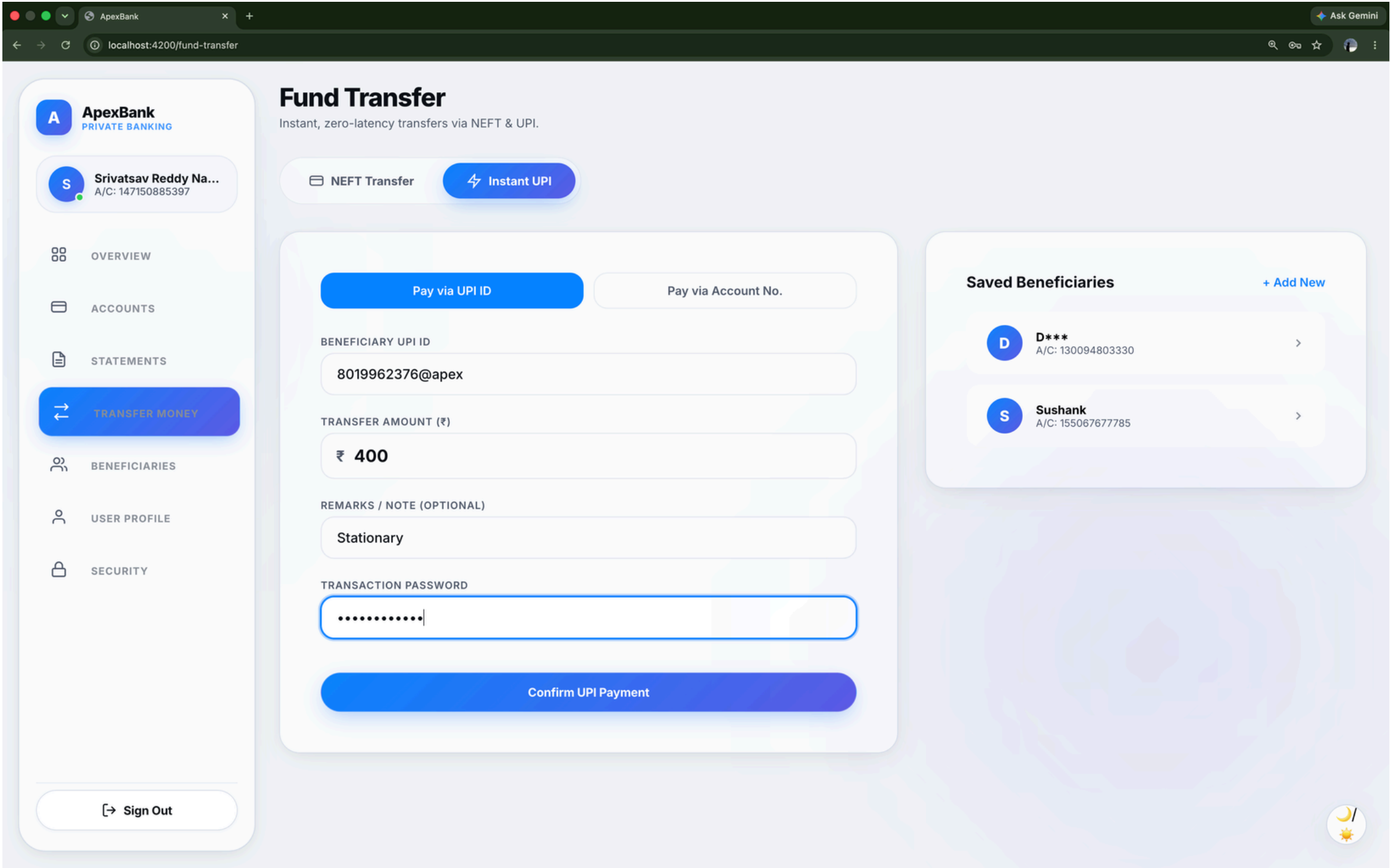
Screen Highlights

- From / To date range filter
- Totals: credit inflow, debit outflow, transactions
- Table with mode, direction, remarks and date

FRONTEND ROUTE
`/dashboard/statement`

API ENDPOINTS
GET `/api/accounts/statement`
POST `/api/fund-transfer/statement`

Fund Transfer Page



Screen Highlights

- NEFT Transfer and Instant UPI tabs
- Pay via UPI ID or account number
- Transaction password confirms the payment

FRONTEND ROUTE

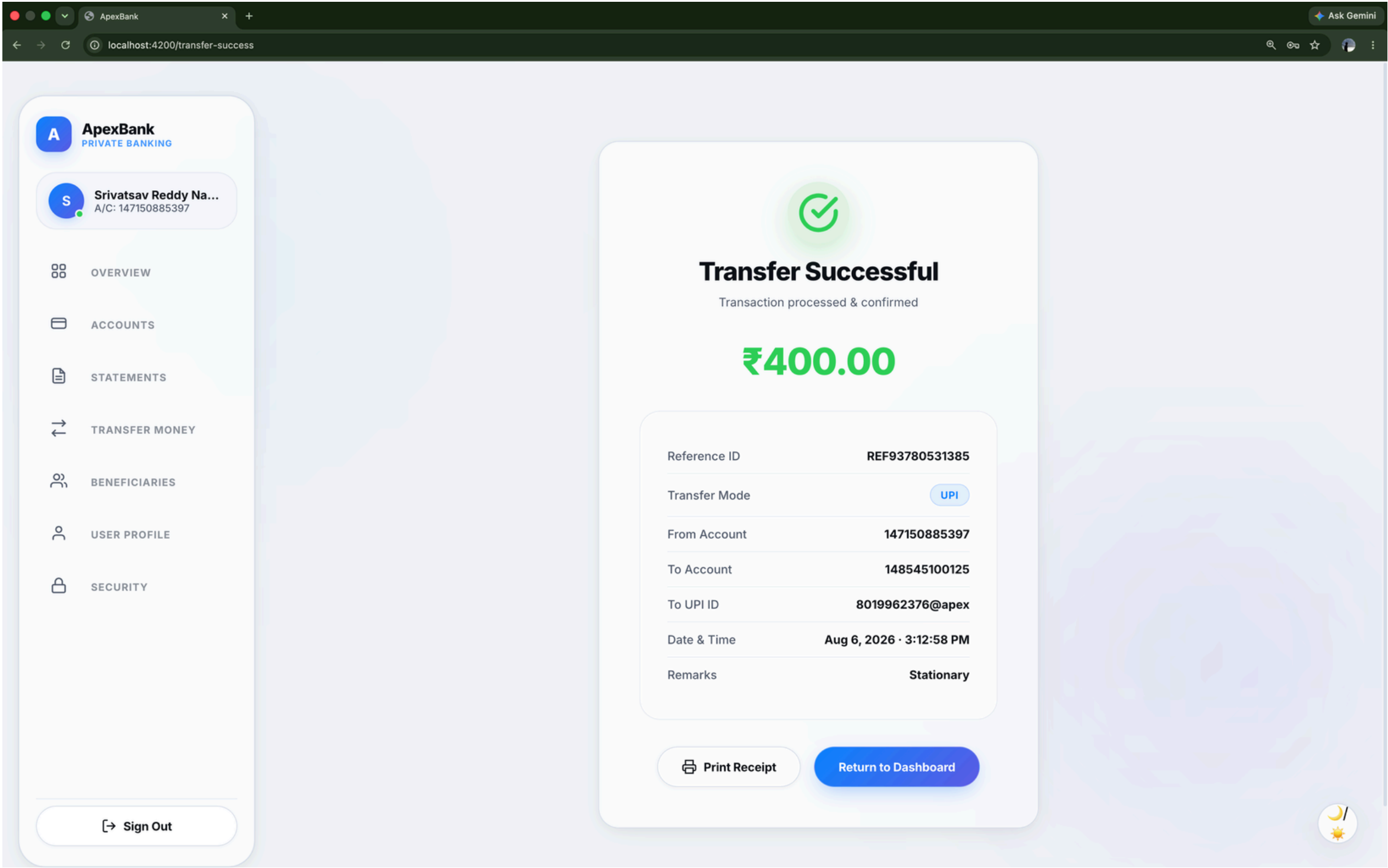
/dashboard/fund-transfer

API ENDPOINTS

POST /api/fund-transfer/neft

POST /api/fund-transfer/upi

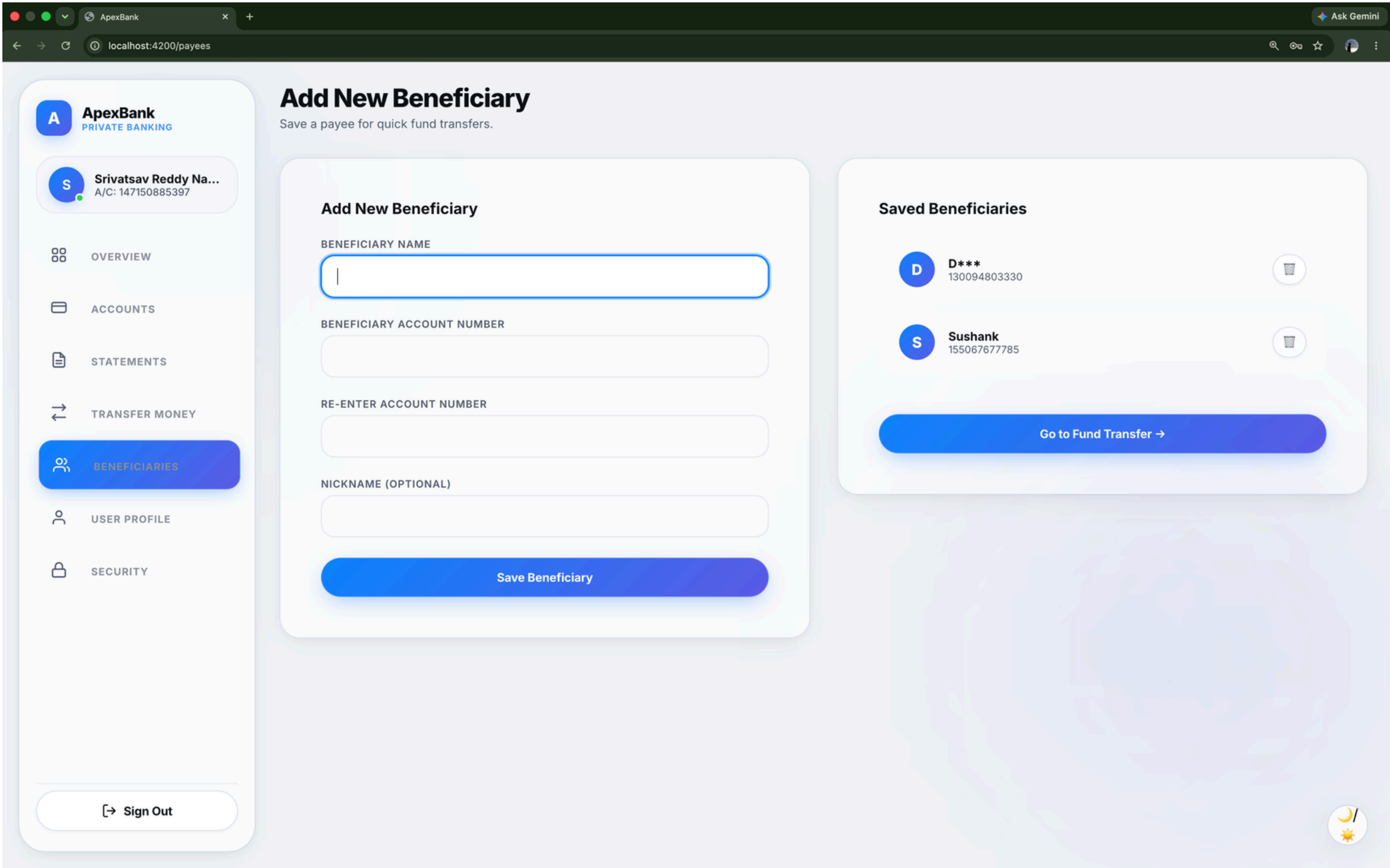
Transaction Success Page



Screen Highlights

- Confirmation message with the transferred amount
- Reference ID, mode, accounts, time and remarks
- Buttons to continue after the transfer

Add Beneficiary Page



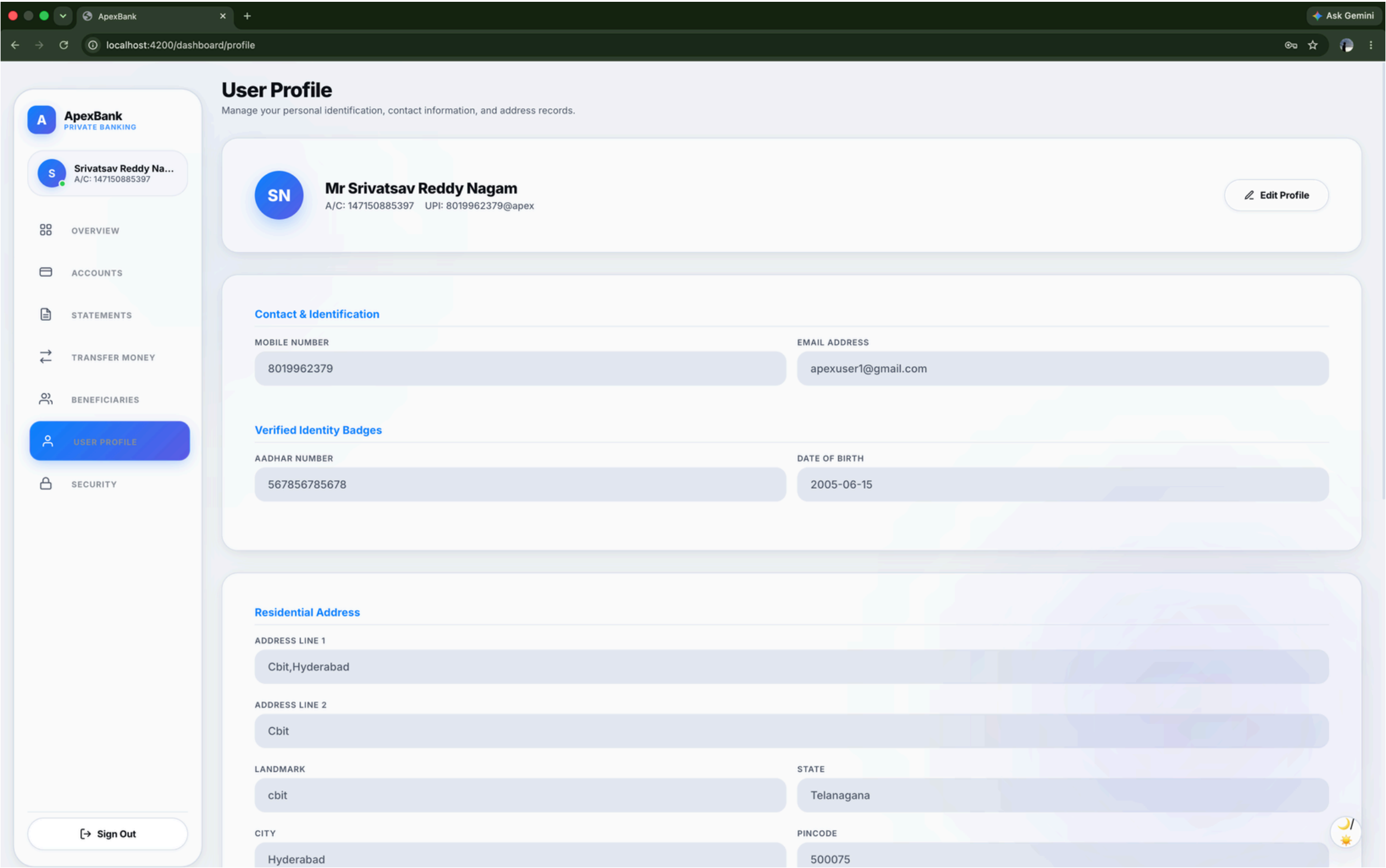
Screen Highlights

- Name, account number (re-entered) and nickname
- Saved beneficiaries list with delete option
- Go to Fund Transfer shortcut

FRONTEND ROUTE
`/dashboard/payees`

API ENDPOINTS
GET `/api/payees`
POST `/api/payees`
DELETE `/api/payees/{id}`

User Profile Page

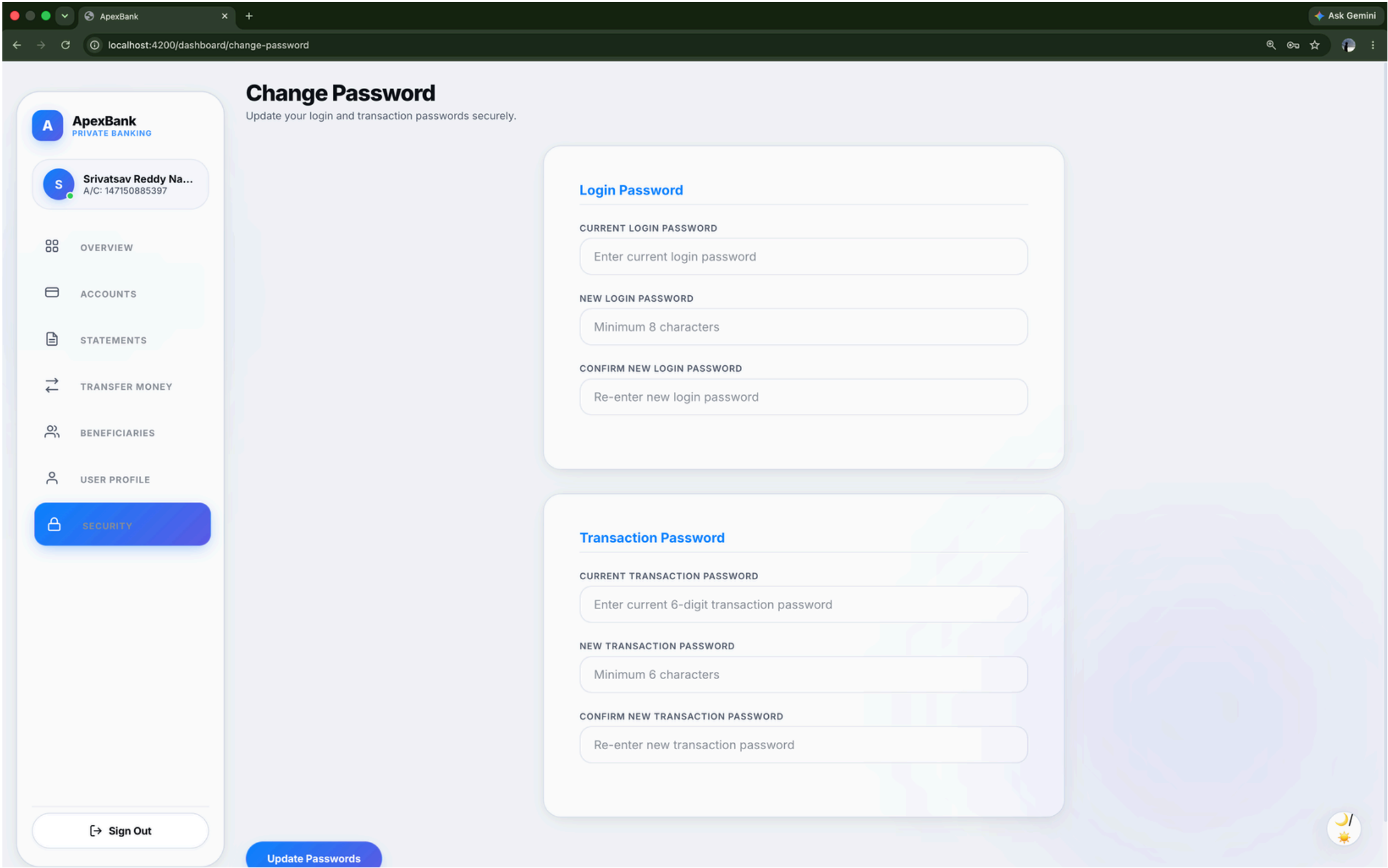


Screen Highlights

- Contact and identification details
- Residential address records
- Profile details can be updated

FRONTEND ROUTE
`/dashboard/profile`

Change Password Page



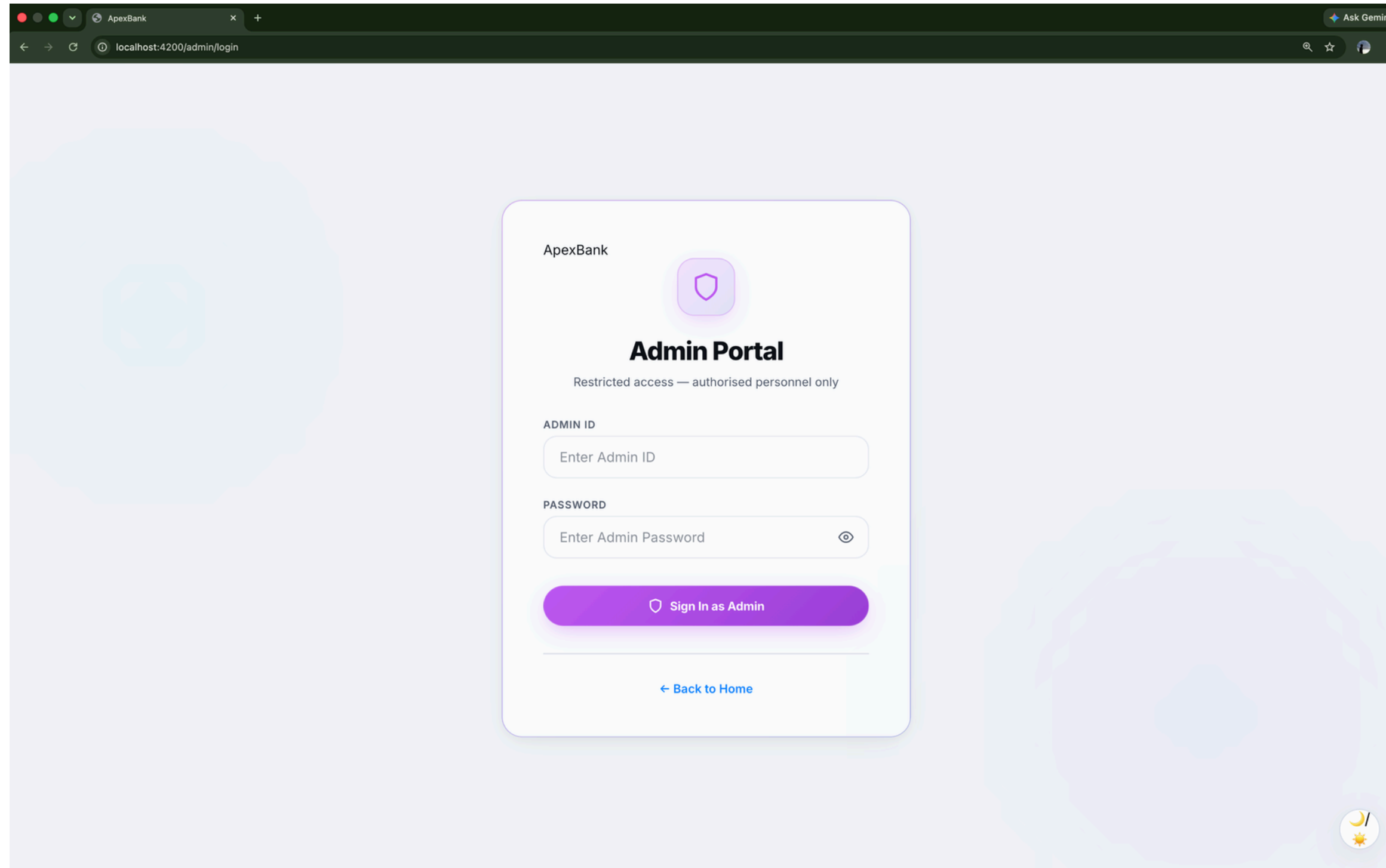
Screen Highlights

- Separate login and transaction password sections
- Current, new and confirm fields
- Update Passwords saves both changes

API ENDPOINTS

POST `/api/auth/change-password`

Admin Login Page

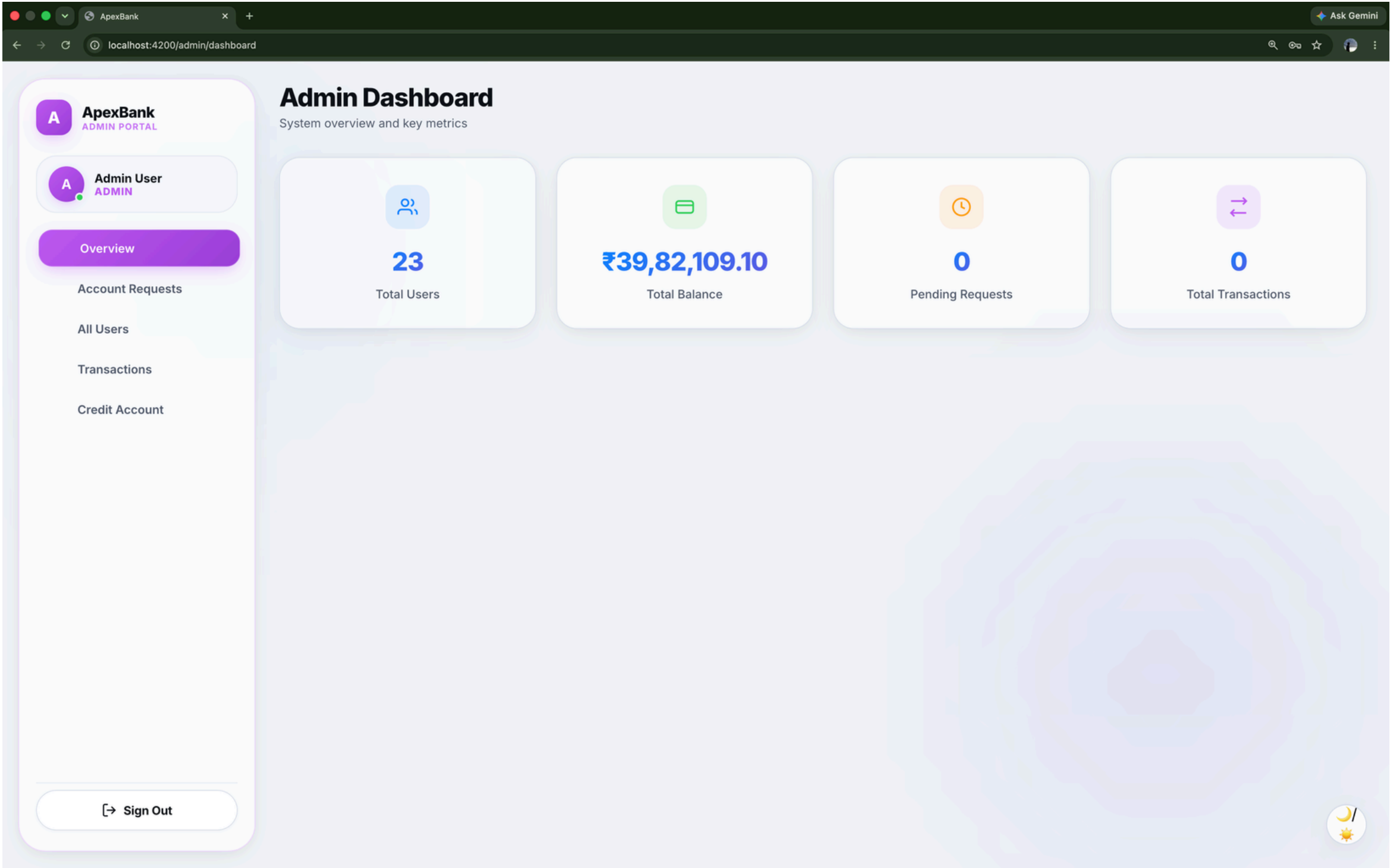


Screen Highlights

- Dedicated login for administrators
- Admin ID and password fields
- Sign-in button for the Admin Portal

FRONTEND ROUTE
`/admin/login`

Admin Dashboard Page

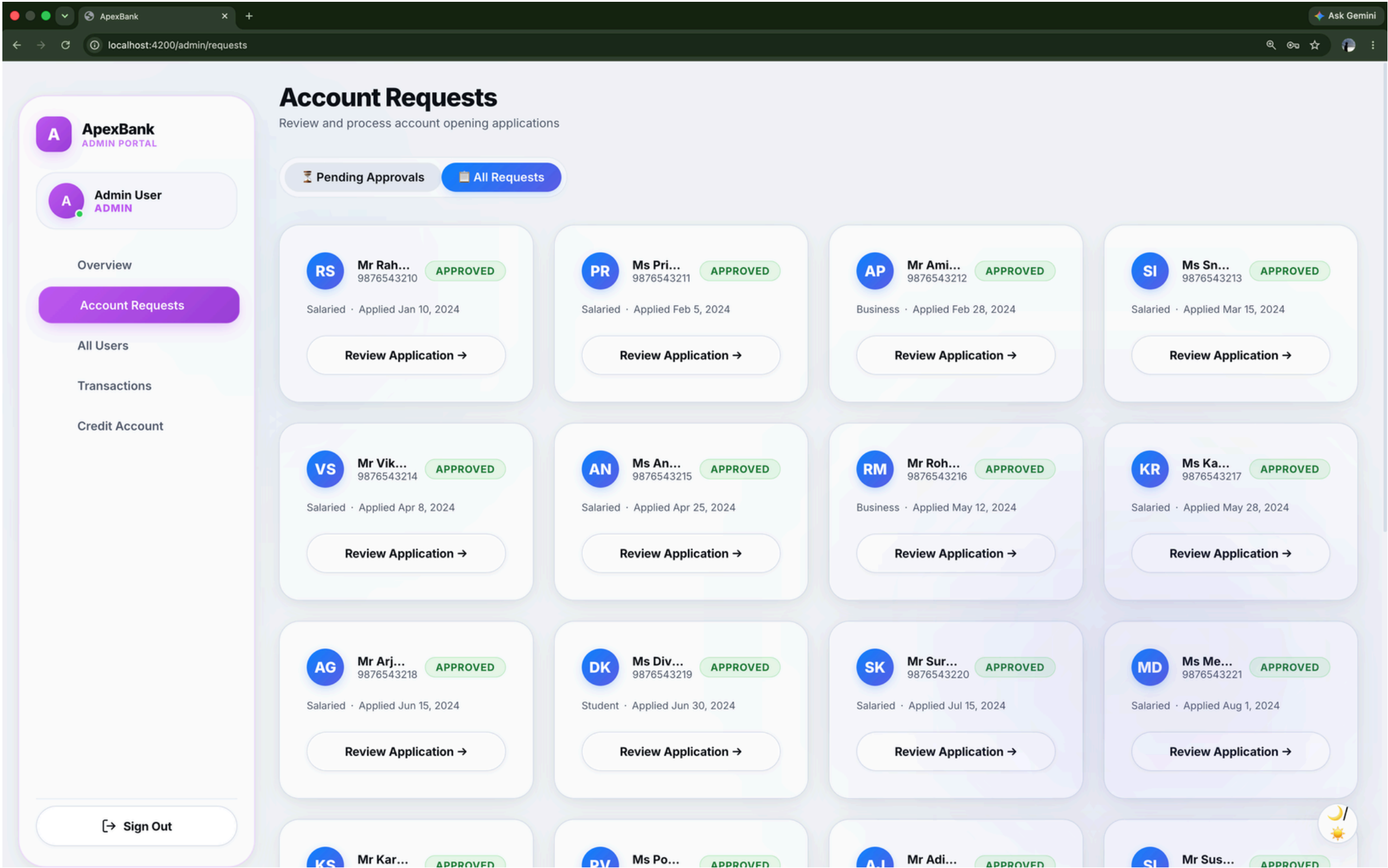


Screen Highlights

- Key metrics: users, total balance, pending requests, transactions
- Side menu: requests, users, transactions, credit

FRONTEND ROUTE
`/admin/dashboard`

Account Requests Page



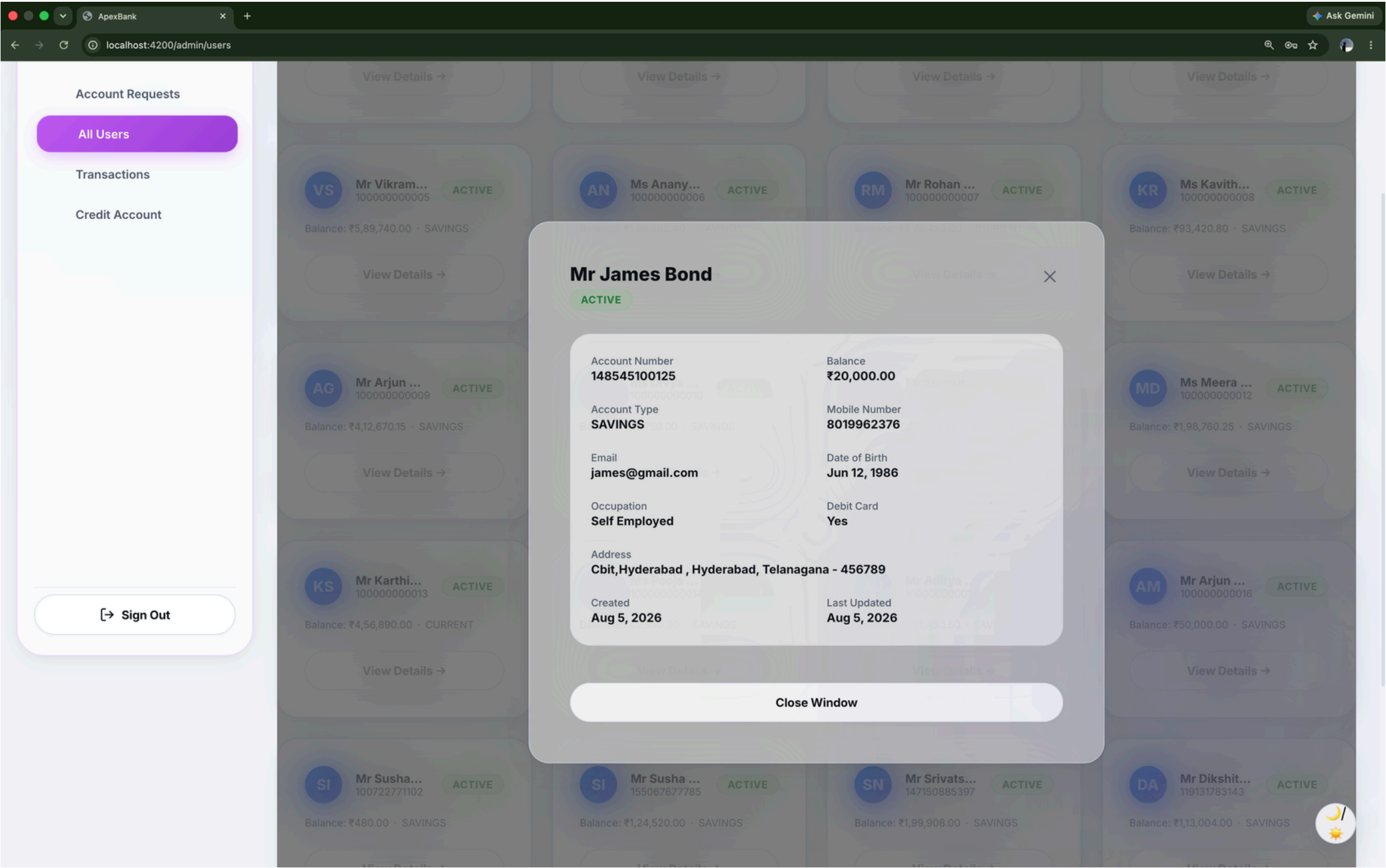
Screen Highlights

- Pending Approvals and All Requests tabs
- Applicant cards with status and occupation
- Review Application to approve or reject

FRONTEND ROUTE
`/admin/requests`

API ENDPOINTS
GET `/api/admin/requests`
POST `.../{id}/approve`
POST `.../{id}/reject`

User Details Page

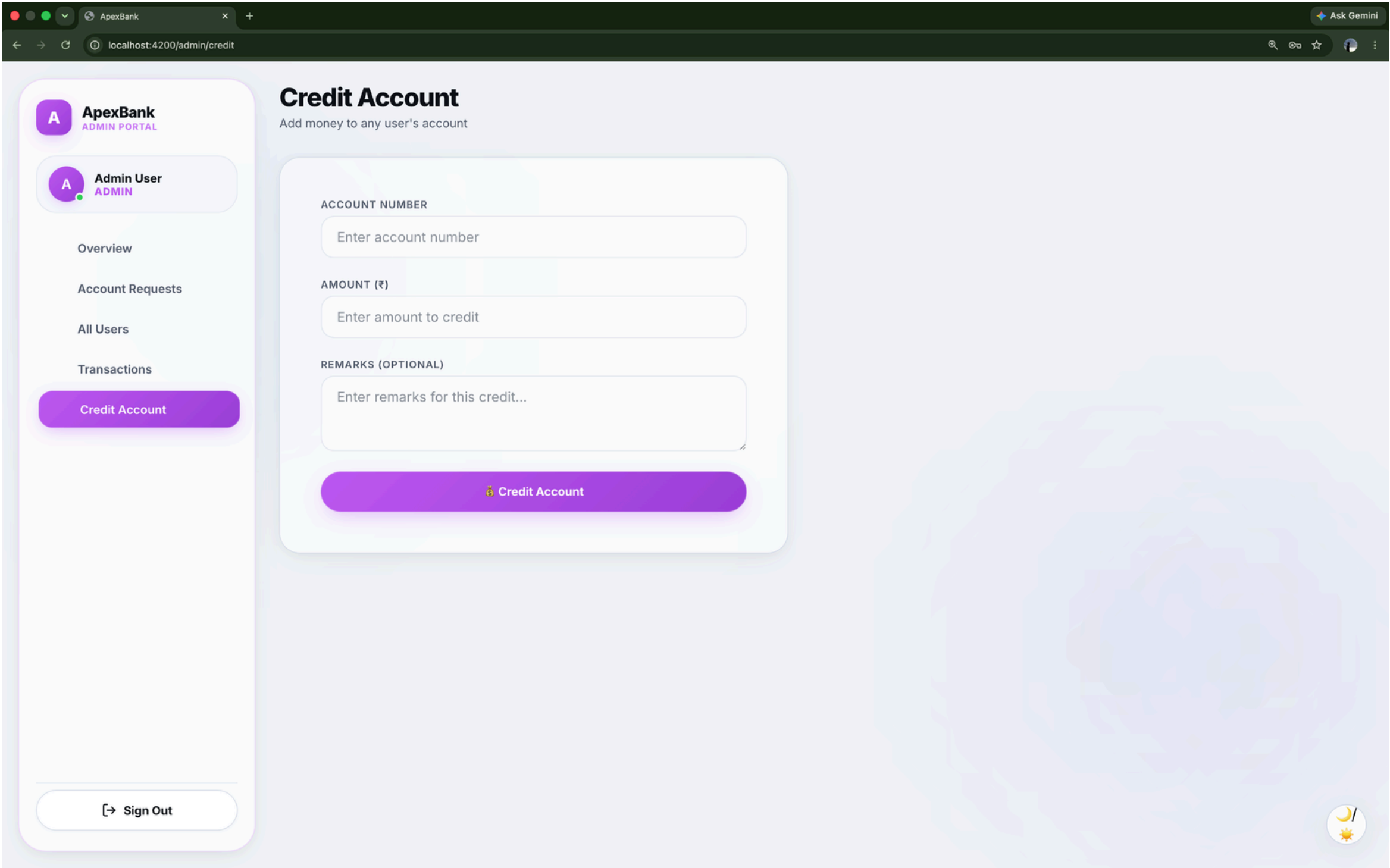


Screen Highlights

- All Users list with active status and balance
- Detail window: account, contact and address
- Close Window returns to the list

FRONTEND ROUTE
`/admin/users`

New Users Amount Credit Page

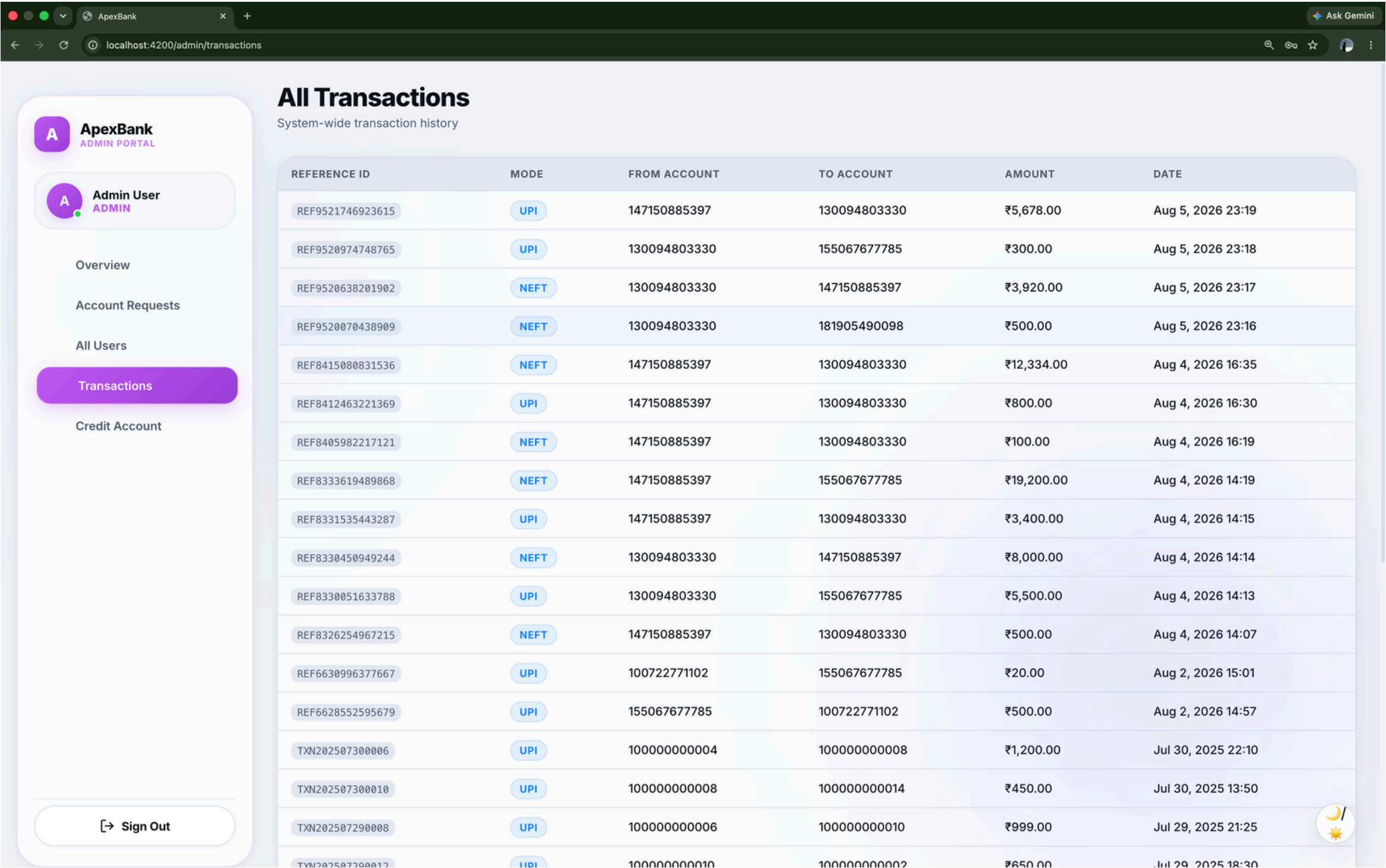


Screen Highlights

- Account number, amount and optional remarks
- Credit Account adds money to the selected user

FRONTEND ROUTE
`/admin/credit`
API ENDPOINTS
`POST /api/admin/credit`

Transactions Tracking Page

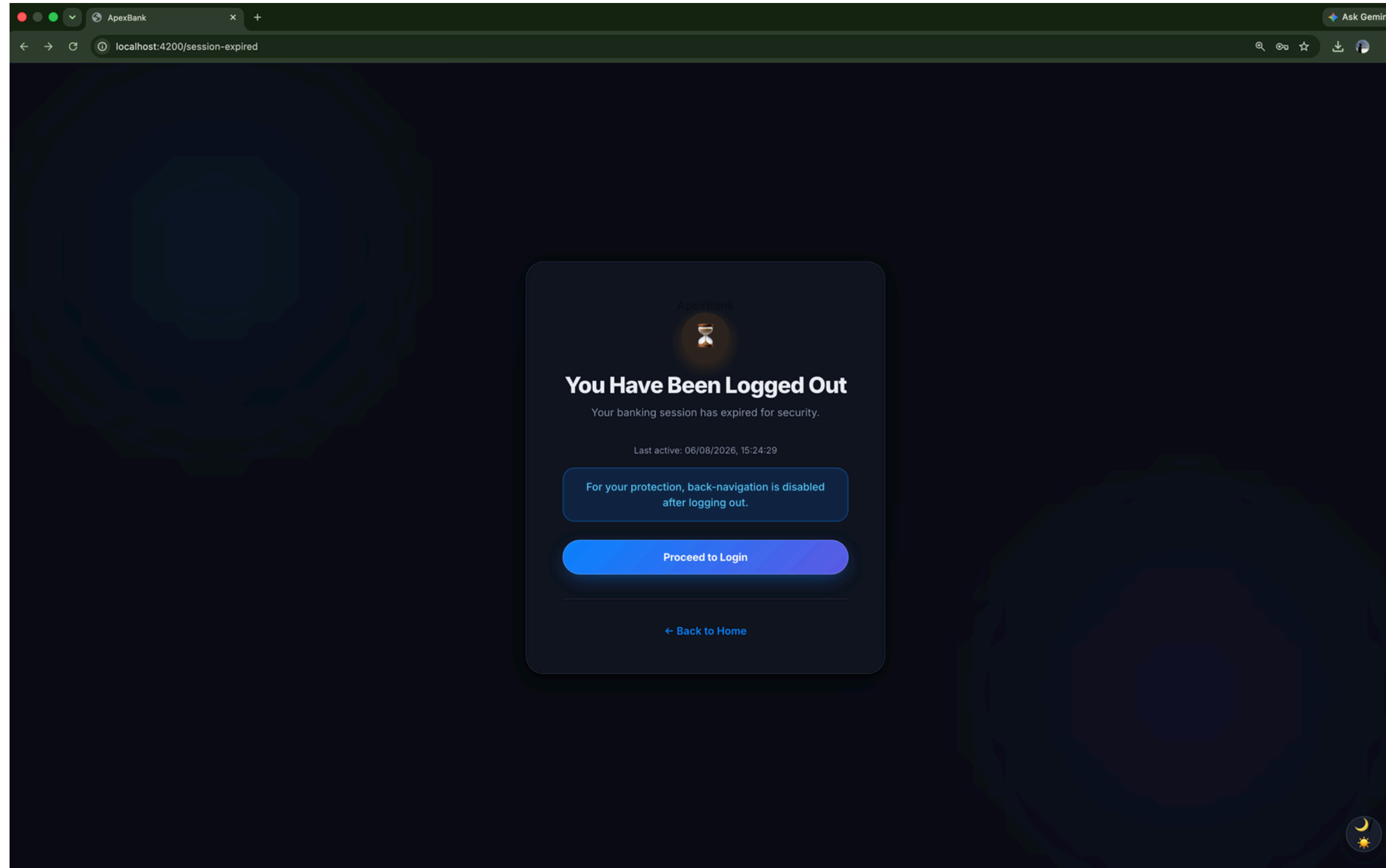


Screen Highlights

- System-wide history of all transactions
- Reference ID, mode, from / to account, amount, date

FRONTEND ROUTE
`/admin/transactions`

Logout Page



Screen Highlights

- Confirmation that the user has been logged out
- Options to log in again or return to the home page

Future Scope

1

Live OTP Delivery

Integrate real SMS / Email gateways for live OTP delivery, replacing the current simulated OTP system.

2

Mobile Applications

Add Android and iOS mobile applications.

3

Biometric 2FA

Add biometric-based two-factor authentication for enhanced accessibility and security.

4

AI Fraud Detection

Introduce AI-based fraud detection to monitor transactions.

5

Real-Time Alerts

Flag suspicious activity in real time.

THANK YOU

Questions & Discussion

ApexBank — Online Banking Portal

Capstone Project

Ade Dikshith

Nagam Srivatsav Reddy

Indroji Sai Sushank

